Reading guide

The halo, the boundary protocol of the section-based controller

Companion to `Chapters/4.Implementation.tex` (`eq:halo`)

This guide explains how a section-based controller sees across the edge of its own zone. It adds no new model. Everything here points back to `eq:halo` in the Method chapter and to the row "zone-local state, boundary protocol" of `tab:information`. The design is logged as `[A6]` in `thesis-progress/temp/GAPS.md`.

Contents

1 The problem the halo solves 1

2 The definition 1

3 How it works, step by step 2

4 A small example 2

5 Why it is built this way 2

6 What the halo does *not* carry 3

7 Summary in five lines 3

1 The problem the halo solves

A section-based controller cuts the warehouse into zones $Y_1, \dots, Y_p$ and runs one network per zone, all with the same weights. If each network saw only its own zone, an agent standing right at a zone edge would be blind to everything one step across it. It could walk into an agent it had no way of seeing, and its crowding count $\delta_i(t)$ would stop at the edge, so the same number would mean something different here than under the other two controllers.

The halo gives each zone a narrow, read-only view across its edge.

2 The definition

$$H_q = \{\, v \notin Y_q : d_G(w, v) \le d_{\text{obs}} \text{ for some } w \in Y_q \,\}.$$

In words, the halo of zone Y_q is every vertex *outside* the zone that is within d_{obs} hops of some vertex *inside* it. It is a band of the neighbouring zones, exactly as deep as an agent's field of view.

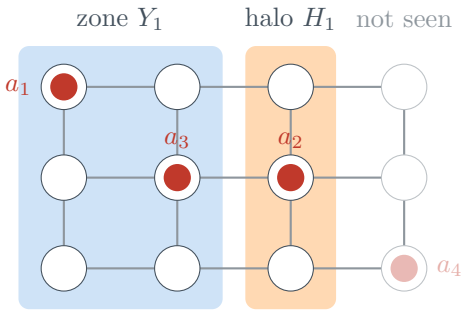
Key idea. The halo is the boundary protocol. In `tab:information`, "zone-local state, boundary protocol" means the zone itself plus its halo, nothing more.

3 How it works, step by step

1. **The network runs on the zone plus the halo.** Zone Y_q's network takes $Y_q \cup H_q$ as its graph.
2. **The halo is read-only.** Halo vertices carry only their role (storage, delivery, endpoint, transit) and whether an agent is standing there. They have no action heads and add nothing to the loss. The zone sees across its edge but never decides there.
3. **A zone owns the agents inside it.** It chooses an action only for agents with $\ell_i(t) \in Y_q$. An agent standing in the halo is visible, but it belongs to the neighbouring zone.
4. **Handover.** When an agent steps into the next zone, that zone takes ownership on the same timestep. Every zone uses the same weights, so only the agent's context changes, not the policy that decides for it.
5. **Conflicts across the edge.** Two zones can each propose a move onto the same boundary vertex. The shared conflict-resolution operator removes that clash, the same way it removes any other. That operator is part of the environment, not part of any controller's window, so using it at a boundary does not let a zone see more.

4 A small example

Take a graph like the small one of `fig:controllers-example`, three rows and four columns. Let zone Y_1 be the left two columns and let $d_{\text{obs}} = 1$. The halo H_1 is then the third column, one hop past the edge. The fourth column is two hops away and is not in the halo.



- Y_1's network decides for a_1 and a_3, the agents inside the zone.
- It sees that the halo vertex next to a_3 is occupied, by a_2. So a_3's crowding count includes a_2, exactly as it would under the decentralised or centralised controller. But Y_1 does not choose a_2's action. a_2's own zone does.
- It does not see a_4, two hops past the edge.
- If a_3 and a_2 both try to move onto the same vertex, the conflict-resolution operator settles it.

5 Why it is built this way

Depth equals d_{obs}. A boundary agent's field of view reaches exactly into the halo, never beyond it, so $\delta_i(t)$ means the same thing under every architecture. It also needs no new parameter.

Direction still works across zones. The distance label η_i uses distances over the whole graph, so an agent heading for a goal in another zone still knows which way it lies. That is the agent's own task information, the same as for the decentralised controller, not information about the other zone.

A continuum of windows. With one zone ($p = 1$) the halo is empty and the section-based controller is exactly the centralised one. With many small zones, each network sees little more than its agents' fields of view, close to the decentralised controller.

It replaced a planning rule. The earlier rule, "yield at the boundary to whichever zone has fewer active agents", compared planned routes. A learned policy acts one step at a time and has no route to compare, so it needed a rule that works per step. Running a graph network on a zone plus a halo is the standard way to run it on a partitioned graph.

6 What the halo does *not* carry

Only roles and occupancy cross the edge. In particular a zone does not learn, from the halo, where an agent across the boundary is heading. Inside its own zone it needs no messages at all, because all of its agents are in the same graph pass, so learned information already flows between them.

Noted variant, not adopted. The halo could be kept and extended with learned messages from the agents standing in it, the same kind of message the decentralised controller uses. That would not remove the boundary protocol, it would widen it. It was not adopted, because the section-based window would then depend on the network's weights, move toward the centralised one, and mix a communication effect into the section-based results that RQ5 is meant to keep separate. Replacing the halo by messages alone is ruled out, since boundary crowding counts need the halo's occupancy. The variant is recorded under `[A6]` in `GAPS.md` and in `message_design.md`.

7 Summary in five lines

1. The halo is the band of vertices within d_{obs} hops outside a zone.
2. A zone's network runs on the zone plus its halo, and the halo is read-only.
3. A zone decides only for the agents inside it, and ownership passes on the step an agent crosses.
4. Conflicts across an edge are settled by the shared conflict-resolution operator.
5. Depth d_{obs} keeps crowding comparable across architectures, and one zone gives back the centralised controller.